DAA Assignment 1

Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.

Assignment Title: Design, Analyze and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm **5 Marks**

Design and propose at least three meaningfully different algorithmic strategies for locating a given registration number.

The three approaches must represent distinct algorithmic strategies. Iterative and recursive implementations of the same underlying algorithm will not be considered as two different approaches.

Your proposed approaches must include:

At least one iterative approach

At least one recursive approach

A clear explanation of the basic idea behind each approach

Pseudocode or an appropriate flowchart for each approach

A brief justification for selecting each approach for investigation

Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to design and select the approaches independently. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

ANSWER – TASK 1

Three distinct strategies are proposed: iterative linear search, recursive binary search, and hash-table lookup. They differ in how they organise and inspect the search space. Binary search requires sorted data, while hashing requires building a lookup index.

Approach 1: Iterative Linear Search

- Basic idea: start from the first record and compare its Registration Number with the target.
- Continue one record at a time until a match is found or the list ends.
- It does not require sorting and is suitable for small or frequently changing unsorted lists.

ALGORITHM Linear Search(A, target)

```
INPUT: A – array of registration numbers; target – number to find
OUTPUT: index of target, or NOT_FOUND
BEGIN
```

```
    FOR i ← 0 TO length(A) - 1 DO
    IF A[i] = target THEN
    RETURN i
```

END IF

END FOR

```
    RETURN NOT_FOUND
    END
```

Approach 2: Recursive Binary Search

- Basic idea: keep the records sorted by Registration Number.
- Compare the target with the middle value.
- If it is smaller, search only the left half; if larger, search only the right half. Each call reduces the remaining search interval by about half.

ALGORITHM RecursiveBinarySearch(A, low, high, target)

```
    INPUT: sorted array A, boundaries low and high, target
    OUTPUT: index of target, or NOT_FOUND
    BEGIN
    IF low > high THEN
    RETURN NOT_FOUND
```

END IF

mid ← FLOOR((low + high) / 2)

```
    IF A[mid] = target THEN
    RETURN mid
    ELSE IF target < A[mid] THEN
    RETURN RecursiveBinarySearch(A, low, mid - 1, target)
    ELSE
    RETURN RecursiveBinarySearch(A, mid + 1, high, target)
```

END IF

```
    END
```

Approach 3: Hash-Table Lookup

- Basic idea: store each registration number as a key in a hash table.
- A hash function maps the key to a table position.
- During a search, the same hash function locates the likely position, and the key is checked there.
- Collisions are handled using chaining or open addressing.

ALGORITHM BuildHashIndex(records)

```
INPUT: records containing unique registration numbers
OUTPUT: hash table H
BEGIN
CREATE empty hash table H
FOR each record r in records DO
INSERT r.registration_number INTO H
```

END FOR

```
RETURN H
END
```

ALGORITHM HashSearch(H, target)

```
BEGIN
IF target is present as a key in H THEN
RETURN H[target]
ELSE
RETURN NOT_FOUND
```

END IF

```
END
```

Strategy	**Design type**	**Preprocessing**	**Expected search cost**	**Reason for selection**
Linear search	Iterative sequential scan	None	O(n)	Simple baseline; works on unsorted

				and changing data
Binary search	Recursive divide-and-conquer	Sort records: O(n log n)	O(log n)	Shows benefit of repeatedly halving a sorted search space
Hash table	Direct key-based indexing	Build index: expected O(n)	Expected O(1)	Suitable when many lookups use a stable dataset

- The strategies are meaningfully different: linear search checks entries sequentially, binary search eliminates half of the remaining interval, and hashing uses a key-to-location index.
- The final choice will be based on total workload, including preprocessing and updates.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the basic operation that predominantly determines its execution time.

Discuss:

What the basic operation is

Where it occurs in the algorithm

Why it is appropriate to use this operation for efficiency analysis

How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to relate the selected operation to the behaviour of the algorithm.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

ANSWER – TASK 2

- A basic operation is the operation repeated most often and therefore strongly influences the running time.
- For this student-record problem, the key operation is a comparison or key lookup involving the Registration Number.

Algorithm	Basic operation	Where it occurs	Why it is suitable
Linear search	A[i] == target	Inside the loop for each record	Every examined record needs a key comparison
Recursive binary search	A[mid] == target and target < A[mid]	At the middle element of each recursive call	The comparison decides which half can be discarded
Hash lookup	Hash computation and key equality check	When mapping target to a bucket and checking collisions	Bucket access is expected constant time; collision checks determine extra work

- How the count changes with n
- Linear search: the number of comparisons grows in direct proportion to the number of records. A missing key requires n comparisons.
- Binary search: each comparison reduces the remaining interval to about half. The number of levels grows logarithmically.
- Hash lookup: the expected number of bucket operations remains nearly constant when the hash function distributes keys well and the load factor is controlled. Poor collision behaviour can increase the number of comparisons.
- For fair analysis, the hash-table build operation must also be counted. Searching may be very fast, but the index takes time and memory to create. The basic operation is therefore considered separately for index construction and for each later lookup.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

The input size (n)

The number of times the basic operation executes

How the operation count changes with (n)

The mathematical expression representing the running time

Any preprocessing cost involved

The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning leading to the complexity result.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

ANSWER – TASK 3

1. Linear Search

- Let n be the number of records.
- If the target is at position k (using positions 1 to n), the algorithm performs k key comparisons.
- If the target is absent, it performs n comparisons.
- Therefore $T(n) = c_0 + c_1k$ for a successful search and $T(n) = c_0 + c_1n$ for an unsuccessful search.
- For a uniformly likely successful position, the average comparison count is $(1 + 2 + \dots + n) / n = [n(n+1)/2] / n = (n+1)/2$. Thus average search work is $\Theta(n)$.

2. Recursive Binary Search

- Let n be the number of sorted records. One call performs a constant number of comparisons and continues with at most n/2 records.
- The recurrence is $T(n) = T(n/2) + c$, with $T(1) = c$.
- After k levels, the remaining size is $n/2^k$. Stop when $n/2^k \leq 1$.
- This gives $2^k \geq n$, so $k \geq \log_2 n$. There are about $\log_2 n$ levels, each doing constant work; therefore $T(n) = \Theta(\log n)$.
- Preprocessing is sorting. A comparison-based sort such as merge sort costs $\Theta(n \log n)$.
- If sorting is performed for every single search, total cost is $\Theta(n \log n + \log n) = \Theta(n \log n)$. If the data is sorted once and many searches follow, the sorting cost can be shared across all queries.

3. Hash-Table Lookup

- Building a hash table for n unique keys takes expected $\Theta(n)$ time when insertion and collision handling are expected constant time. One later lookup takes expected $\Theta(1)$ time.

- For q searches after one build, expected total work is $\Theta(n + q)$.
- In the worst case, many keys may collide into one bucket. A lookup can then require $\Theta(n)$ comparisons.
- A well-designed hash function, suitable table capacity and controlled load factor make this worst case unlikely, but do not remove the theoretical possibility.

Approach	Preprocessing	One search	Total for q searches
Linear	0	$\Theta(n)$ worst case	$\Theta(qn)$
Binary	$\Theta(n \log n)$ sort	$\Theta(\log n)$	$\Theta(n \log n + q \log n)$
Hash table	Expected $\Theta(n)$ build	Expected $\Theta(1)$	Expected $\Theta(n + q)$

- The table shows why preprocessing must be included. For a single search, linear search may be simpler.
- For repeated searches, the one-time cost of sorting or hashing can be justified because it is reused.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

What input condition produces the best case

What input condition produces the average case

What input condition produces the worst case

How the number of operations changes in each case

Why these differences occur

Do not simply provide a table containing complexity values. You are expected to explain the conditions that lead to each case.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

ANSWER – TASK 4

Algorithm	Best case	Average case	Worst case
Linear search	1 comparison: target is first	(n+1)/2 comparisons for uniformly placed successful target; about n/2	n comparisons: target last or absent
Recursive binary search	1 comparison: target is at middle	Θ(log n) comparisons	Θ(log n), including an unsuccessful search
Hash table	Expected near 1 bucket access and few checks	Expected Θ(1) with good distribution	Θ(n) if all keys collide

Explanation of the cases

- Linear search has different cases because the algorithm stops as soon as it finds the target.
- A first-position match is fastest.
- A last-position or missing target forces a complete scan.
- If successful positions are equally likely, the average is (n+1)/2 comparisons.
- Binary search has logarithmic behaviour in all cases except the exact middle hit, which takes one comparison.
- Each unsuccessful step discards roughly half the remaining range, so even the worst case has only about $\log_2 n$ levels.
- Hash-table performance depends on the distribution of keys.
- With a good hash function, keys are spread across buckets and only a small number of entries need checking. If collisions become concentrated, lookup can degrade.
- The average-case claim is therefore conditional on reasonable hashing and load-factor management.
- For the university, unsuccessful searches also matter: a registration number may be typed incorrectly or may not exist.

- Linear search pays for the entire list in this case, while binary search and a hash index can establish absence much faster.

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant

How each algorithm behaves as (n) becomes very large

How the growth rates of the proposed approaches differ

What the growth behaviour means when the dataset increases from 1,000 to 1,000,000+ records

You are expected to interpret the asymptotic behaviour in the context of the given application, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

ANSWER – TASK 5

- Asymptotic notation describes how operation counts grow as n becomes large.
- It ignores constant factors and lower-order terms, allowing the three designs to be compared independently of a particular computer.

Algorithm	Best-case bound	Average / expected bound	Worst-case bound	Meaning for large n
Linear search	$\Omega(1)$	$\Theta(n)$	$O(n)$	Work grows with the number of records
Binary search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$	Search growth is slow after sorting
Hash lookup	Expected $\Omega(1)$	Expected $\Theta(1)$	$O(n)$	Usually near constant, but collision worst case is linear

- When n increases from 1,000 to 1,000,000, linear search can require one thousand times as many comparisons in a full scan.
- Binary search grows from about 10 comparisons to about 20 comparisons.
- Expected hash lookup remains approximately constant in terms of table operations, assuming good hashing and resizing.
- Big-O gives an upper growth bound, Omega gives a lower bound, and Theta gives a tight bound.
- For linear search, the worst case is $\Theta(n)$.
- For binary search, the worst case is $\Theta(\log n)$.
- For hash lookup, expected $\Theta(1)$ is a practical average-case statement, while worst-case $O(n)$ must still be acknowledged.
- The notation does not measure all real-world costs. Cache behaviour, memory allocation, disk access, network latency and implementation details can affect observed time.
- Therefore asymptotic analysis should be combined with operation counts and repeatable experiments.

TASK 6 – Formulate the Recurrence Relation **3 Marks**

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

The input size of the original problem

The size of the recursive subproblem

The number of recursive calls

The work performed outside the recursive call

The base case

How these components lead to the recurrence relation

The recurrence must be derived from your algorithm, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

ANSWER – TASK 6

- The recursive approach selected in Task 1 is recursive binary search.
- It operates on a sorted array and examines the middle record in each call.

Recurrence component	**Binary-search description**
Original input size	n registration numbers in the current search interval
Subproblem size	At most floor(n/2) records
Number of recursive calls	One recursive call after the comparison, unless the key is found
Work outside recursion	Middle-index calculation and a constant number of comparisons
Base case	low > high (empty interval), or target equals middle
Recurrence	$T(n) = T(\lfloor n/2 \rfloor) + c$ for $n > 1$; $T(0) = c_0$

- The recurrence comes directly from the algorithm.
- It does not create two recursive calls: only one half is searched.
- The other half is discarded because the array is sorted and the comparison proves that the target cannot be in that half.
- The middle comparison and boundary check take constant time, represented by c.
- The remaining subproblem contains approximately half of the previous elements. This gives $T(n) = T(n/2) + c$.
- The recurrence applies to the search phase only.

- Sorting the original records is a separate preprocessing step with cost Θ(n log n), and it must be added when calculating the complete system cost.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

Substitution method

Iteration method

Master Theorem

Another appropriate analytical technique

Discuss:

Why the selected method is applicable

The steps involved in solving the recurrence

The resulting complexity

What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to show the analytical process, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

ANSWER – TASK 7

- We solve T(n) = T(n/2) + c, where c is constant and T(1) = d.
- The iteration method is suitable because the recurrence repeatedly reduces the problem size by half.

Iteration steps

T(n) = T(n/2) + c

= T(n/4) + c + c

= T(n/8) + 3c

= T(n/2^k) + kc

Stop when n/2^k = 1

Therefore 2^k = n

Taking logarithms: k = log2(n)

T(n) = T(1) + c log2(n)

= d + c log2(n)

= Θ(log n)

- The result means that doubling the number of sorted records adds approximately one comparison level, not twice the search work.
- For example, a million records need about 20 halving steps because 2^{20} is approximately one million.
- The Master Theorem also applies with a = 1, b = 2 and f(n) = Θ(1). Since n^(log_b a) = n^0 = 1, this is the balanced case and gives Θ(log n).
- The iteration derivation is shown because it makes repeated halving visible.
- This solution describes the recursive search only.
- If the records are not already sorted, sorting cost must be added: Θ(n log n) + Θ(log n) = Θ(n log n) for a one-time sort and one search.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

All algorithms produce the correct result

The same or equivalent datasets are used for comparison

Testing conditions are kept reasonably consistent

Appropriate test cases are included

The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is not merely to demonstrate working code, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

ANSWER – TASK 8

- The following Python program implements the three strategies.
- It uses the same list of unique registration numbers for all methods.
- Binary search receives a sorted copy, and the hash method builds a dictionary index.
- **def linear_search(records, target):**
- **for i, value in enumerate(records):**
- **if value == target:**

1. return i
2. return -1

- **def recursive_binary_search(records, target, low=0, high=None):**
- **if high is None:**

1. high = len(records) - 1

- **if low > high:**

1. return -1
2. mid = (low + high) // 2

- **if records[mid] == target:**

1. return mid

- **if target < records[mid]:**

1. return recursive_binary_search(records, target, low, mid - 1)
2. return recursive_binary_search(records, target, mid + 1, high)

- **def build_hash_index(records):**

1. return {value: i for i, value in enumerate(records)}

- **def hash_search(index, target):**

1. return index.get(target, -1)

- **def run_tests():**

1. data = [1042, 1008, 1099, 1025, 1070]
2. sorted_data = sorted(data)
3. index = build_hash_index(data)
4. assert linear_search(data, 1042) == 0
5. assert linear_search(data, 1070) == 4
6. assert linear_search(data, 9999) == -1
7. assert recursive_binary_search(sorted_data, 1008) == 0
8. assert recursive_binary_search(sorted_data, 1099) == 4
9. assert recursive_binary_search(sorted_data, 9999) == -1
10. assert hash_search(index, 1042) == 0
11. assert hash_search(index, 9999) == -1
12. assert linear_search([], 1000) == -1
13. assert recursive_binary_search([], 1000) == -1
14. print("All test cases passed.")

- **if __name__ == "__main__":**

1. run_tests()

Test cases

Test case	Input condition	Expected result	Purpose
T1	Target is first element	Correct index	Best case for linear search
T2	Target is last element	Correct index	Worst successful linear search
T3	Target is absent	-1 / NOT_FOUND	Unsuccessful search
T4	Empty list	-1 / NOT_FOUND	Boundary condition
T5	Single record, matching key	Index 0	Smallest valid input
T6	Single record, different key	-1 / NOT_FOUND	Single-element failure
T7	Duplicate registration number	Not applicable; input validation should reject duplicates	Checks unique-key assumption

- The program uses assertions so that a wrong result causes a test failure.
- The binary-search input is sorted before searching. The hash index is built once and then reused.
- In a real database, the hash index should store the full record or a pointer to the record, not only the array index.

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

Execution time

Number of comparisons

Number of key operations

Input size

Other relevant performance measures, where justified

Present the results using appropriate:

Tables

Graphs

Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

ANSWER – TASK 9

- Operation counts are more reproducible than execution time because they do not depend strongly on processor speed or background applications.
- The following table gives theoretical worst-case comparison counts for the search phase.
- These are calculated values, not measured runtime results.

Number of records (n)	Linear search (missing key)	Binary search (maximum levels)	Hash lookup (expected)
1,000	1,000	10	Expected near 1 bucket access
10,000	10,000	14	Expected near 1 bucket access
100,000	100,000	17	Expected near 1 bucket access
1,000,000	1,000,000	20	Expected near 1 bucket access

- For binary search, ceil($\log_2(n+1)$) is used as a simple upper-level estimate.
- Exact comparison counts can vary slightly with the target position and implementation.
- Hash lookup count depends on collisions and the chosen dictionary implementation.
- **Benchmark program for actual measurements**

1. import random
2. import time

- **def linear_count(a, target):**

1. comparisons = 0

- **for i, value in enumerate(a):**

1. comparisons += 1

- **if value == target:**

1. return i, comparisons
2. return -1, comparisons

- **def binary_count(a, target):**

1. low, high = 0, len(a) - 1
2. comparisons = 0

- **while low <= high:**

1. mid = (low + high) // 2
2. comparisons += 1

- **if a[mid] == target:**

1. return mid, comparisons
2. comparisons += 1

- **if target < a[mid]:**

1. high = mid - 1

- **else:**

1. low = mid + 1
2. return -1, comparisons

- **def measure():**
- **for n in [1000, 10000, 100000, 1000000]:**

1. data = list(range(n))
2. random.shuffle(data)
3. sorted_data = sorted(data)
4. target = -1
5. start = time.perf_counter()
6. linear_result = linear_count(data, target)
7. linear_time = time.perf_counter() - start
8. start = time.perf_counter()
9. binary_result = binary_count(sorted_data, target)
10. binary_time = time.perf_counter() - start
11. start = time.perf_counter()
12. index = {value: i for i, value in enumerate(data)}
13. build_time = time.perf_counter() - start
14. start = time.perf_counter()
15. hash_result = index.get(target, -1)
16. hash_time = time.perf_counter() - start
17. print(n, linear_result[1], binary_result[1],
18. round(linear_time, 6), round(binary_time, 6),
19. round(build_time, 6), round(hash_time, 6))

- **if __name__ == "__main__":**

1. measure()

- Run the benchmark three times and record the median time.

- Keep the same computer, Python version and test key.
- Include the hash-table build time separately so that the preprocessing cost is not hidden.

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

Whether the observed growth agrees with the theoretical complexity

Which algorithm scales better as (n) increases

Any differences between theoretical prediction and actual execution

Possible reasons for the differences

You are expected to interpret the results, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

ANSWER – TASK 10

- The theoretical analysis predicts linear growth for a full linear scan, logarithmic growth for binary search, and expected constant-time hash lookup after index construction.
- The benchmark should be interpreted against these predictions.

Measure	Theoretical expectation	What to check in the experiment	Reason for possible difference
Linear comparisons	Approximately n for a missing key	Count should double when n doubles	Loop and interpreter overhead affect time, not the comparison count
Binary comparisons	About $\log_2 n$ levels	Count should increase slowly	Exact count depends on target and comparison implementation
Hash lookup	Expected near constant	Lookup time should stay relatively stable	Collisions, resizing, memory/cache effects
Hash preprocessing	Expected $\Theta(n)$	Build time rises with n	Allocation and memory management add overhead

Binary preprocessing	Θ(n log n) sorting	Sort time rises faster than a linear scan	Sorting algorithm and input order affect constants

- A difference between theoretical and observed time does not automatically mean the analysis is wrong.
- Big-O describes growth, not exact seconds.
- For small datasets, a simple linear scan may be faster than building an index because index creation has a fixed cost.
- For reliable comparison, use multiple repetitions, report the median, avoid unrelated programs running during measurement, and keep the input and target conditions consistent.
- Operation counts should be treated as primary evidence, while elapsed time is supporting evidence.
- The expected conclusion is that linear search becomes less suitable as the dataset and request volume increase.
- Binary search and hashing scale better for repeated queries, but the final decision depends on update frequency, memory and operational requirements.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

Which approach you recommend

Why it is more appropriate than the alternatives

Its theoretical efficiency

Its experimentally observed performance

Any preprocessing cost involved

Relevant trade-offs or limitations

Whether the solution remains practical as the database grows

Your recommendation must be based on evidence obtained from your own analysis and experimentation.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

ANSWER – TASK 11

Recommendation: use a hash-table index keyed by the unique Registration Number for the main student-record lookup service. Keep the authoritative student records in the university database and maintain the hash index as a lookup structure.

- Reasons for selecting hashing
- The system performs many searches against a relatively stable set of records.
- Expected lookup time is $\Theta(1)$, which is suitable for frequent registration-number queries.
- The index can be built once and reused for many searches.
- New records are added only once per day, so the index can be updated in batches or rebuilt during a controlled maintenance window.
- The registration number is unique, making it a suitable key for direct lookup.

Criterion	Linear search	Binary search	Hash-table index
Search cost	$\Theta(n)$	$\Theta(\log n)$	Expected $\Theta(1)$
Preprocessing	None	Sort: $\Theta(n \log n)$	Build: expected $\Theta(n)$
Updates	Easy append	May require maintaining	Expected constant-time

		sorted order	insert per new key
Memory overhead	Low	Low to moderate	Additional index memory
Suitability for 50,000 searches/hour	Poor for large n	Good	Very good under normal hashing

- Binary search is a strong alternative when sorted order is already maintained or when ordered range queries are also required.
- Hashing does not naturally return records in sorted order.
- Linear search remains useful for tiny lists, one-off checks or situations where maintaining an index is not worthwhile.
- The recommendation is not based only on Big-O notation.
- It considers the repeated search workload, daily update pattern, preprocessing reuse and memory trade-off.
- The university should monitor index load factor, collision behaviour, memory use and lookup latency.

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records.

However, new student records are added only once per day.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

What has changed from the original problem

Which original assumptions are affected

Why the frequency of searches is now important

Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

Its expected behaviour under 50,000 searches per hour

Whether its preprocessing cost is acceptable under the new conditions

Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

Describe the revised algorithmic strategy

Explain what has changed

Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

Mathematical complexity analysis

Experimental results, where appropriate

Search frequency

Preprocessing cost

Dataset size

Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

ANSWER – TASK 12

A. Impact Analysis (2 Marks)

- The changed requirement specifies 1,000,000 student records and 50,000 search requests per hour.
- This is approximately 13.9 requests per second on average.
- The dataset is large, and searches occur much more frequently than updates.

Original assumption	Changed condition	Design impact
Dataset may range from 1,000 upward	Fixed challenge size: 1,000,000	Large-scale behaviour becomes central
Search workload not fixed	50,000 searches/hour	Lookup cost is multiplied across many requests
Record updates may occur	New records once per day	Index maintenance can be batched

Any suitable strategy can be compared	High read-to-write ratio	Preprocessing is easier to justify

- The frequency of searches matters because a small saving per lookup is multiplied across 50,000 requests every hour.
- The low update frequency matters because the system does not need to rebuild its index after every query or continuously reorder the complete dataset.

B. Evaluation of the Original Design (2 Marks)

- The original recommendation of a hash-table index remains appropriate.
- Building the index requires expected $\Theta(n)$ work, but the index can serve many lookups.
- With one million records and 50,000 searches per hour, the build cost is amortised across a large number of requests.
- A sorted array with binary search is also efficient, but insertion into a contiguous sorted array can require shifting elements.
- Since updates occur daily, this is manageable, but a hash index gives direct expected lookup for exact registration-number searches.
- If ordered reporting is needed, a separate sorted view can be maintained.

C. Adaptation or Redesign (2 Marks)

- Decision: MODIFY the original design by introducing a managed hash index and a daily update process.
- The authoritative database remains the source of truth.
- The index maps each unique registration number to the corresponding record identifier.

ALGORITHM DailyIndexUpdate(database, index)

```
BEGIN
FOR each new student record r added today DO
IF r.registration_number is not already in index THEN
INSERT r.registration_number → r.record_id INTO index
ELSE
```

FLAG duplicate registration number for validation

END IF

END FOR

VERIFY index consistency

PUBLISH updated index

```
END
```

ALGORITHM FindStudent(index, registration_number)

```
BEGIN
```

record_id ← HASH_LOOKUP(index, registration_number)

```
IF record_id is NOT_FOUND THEN

RETURN NOT_FOUND
```

END IF

```
RETURN FETCH_RECORD(record_id)

END
```

- Expected complexity: daily index construction or batch update is expected O(n) for a full build, or expected O(u) for u new records when updating an existing hash index.
- Each exact-key lookup is expected O(1), with O(n) worst case under severe collisions.
- Additional memory of O(n) is required for the index.

D. Evidence-Based Defence (2 Marks)

Evidence	**Result / interpretation**
Dataset size	1,000,000 records
Search rate	50,000 per hour ≈ 13.9 per second on average
Hash index build	Expected O(n), performed once or as a daily batch
Each lookup	Expected O(1)
Daily update	Expected O(u) for u new records
Memory trade-off	O(n) extra index storage
Alternative	Binary search is O(log n), but requires sorted data and ordered maintenance

- The modified hash-index design fits the read-heavy workload.
- It avoids scanning one million records for every request and avoids repeatedly sorting the complete dataset.
- The once-daily update schedule makes index maintenance practical.
- The design should still include duplicate-key validation, backup/recovery, access control and consistency checks.
- A production deployment should confirm the result using real request distributions and measured latency.
- Average request rate is not enough by itself; peak traffic, concurrent access, database response time and failure recovery must also be tested.
- Implementation Notes and Validation Checklist
- The following checklist can be used before submitting the assignment. It connects the algorithm design, mathematical analysis and experimental evidence.

Validation item	**How it is checked**	**Status**
Unique registration numbers	Reject duplicates during data import	To be verified
Linear search correctness	Test first, middle, last and missing key	Covered by test harness
Binary search precondition	Confirm data is sorted before search	Covered by implementation
Recursive base case	Test empty list and missing target	Covered by test harness
Hash index consistency	Compare index keys with database keys	To be verified with real database
Preprocessing cost	Measure sorting or index construction separately	Benchmark provided
Operation counts	Record comparisons and lookup operations	Benchmark provided
Runtime measurements	Repeat runs and report median	Run on local computer
Large dataset behaviour	Test 1,000 through 1,000,000 records	Benchmark provided
Daily update process	Insert new records and verify lookup	To be verified

Viva Preparation – Short Answers

Viva question	**Simple answer**
Why is linear search O(n)?	Because in the worst case it checks every record once.
Why must binary search use sorted data?	The ordering tells the algorithm which half cannot contain the target.
Why is binary search logarithmic?	Each comparison reduces the remaining search interval to about half.
What is the recurrence for recursive binary search?	$T(n) = T(n/2) + c$, with a constant-time base case.
Why is hashing fast on average?	The hash function maps a key directly to a bucket, so only a small number of entries are checked on average.
What is a collision?	A collision occurs when two different keys map to the same bucket.
Why include preprocessing cost?	Because sorting or building an index takes time and resources before searching starts.
Why choose hashing for the changed requirement?	There are many exact-key searches and updates occur only once per day.
What is the main disadvantage of hashing?	It needs extra memory and does not naturally maintain sorted order.
Can hash lookup become O(n)?	Yes. In the worst case, many keys may collide in the same bucket.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3

Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50